

Independent Architecture Investigation

Restaurant Automation SaaS — Repeated Requests / Socket Storm / Remount Symptoms

This is a from-scratch trace of the actual code in the uploaded ZIP (not the Executive Summary). Every claim below is checked against the real files. Where the Executive Summary quoted code that does not exist in the repo, that is called out explicitly.

1. What actually boots

`frontend/index.html` loads `/src/main.tsx` (Vite entry). There is a **second, unused duplicate** at `src/app/main.tsx` that also renders `<AuthProvider><SocketProvider><AppRouter></SocketProvider></AuthProvider>` — dead code, not the real entry, but worth deleting since it can mislead future debugging (it clearly misled the Executive Summary, which describes `AuthProvider.tsx`/`ProtectedRoute.tsx` code that doesn't match what's on disk).

The real entry:

```
tsx

<React.StrictMode>
  <ThemeProvider>
    <QueryProvider>
      <AuthProvider>
        <SocketProvider>
          <AppRouter />
        </SocketProvider>
      </AuthProvider>
    </QueryProvider>
  </ThemeProvider>
</React.StrictMode>
```

`React.StrictMode` is present. In React 18 dev mode this deliberately mounts every component twice and double-invokes every effect (mount → cleanup → mount) to surface unsafe effects. The Executive Summary never mentions this. It is a real, first-order contributor to "sockets connect/disconnect on load" and "console logs fire twice" *in development* — and it's the kind of thing that's easy to mistake for a bug when it's actually React telling you the effect isn't idempotent-safe (see §3).

`app/providers/AuthProvider.tsx` is just `export { AuthProvider, useAuth, ... } from '../..../auth/AuthProvider'` — a re-export, not a second implementation. So all `useAuth()` consumers across the app (30+ files) share one real context defined in `src/auth/AuthProvider.tsx`.

2. Verifying every Executive Summary claim against the real code

#	Claim	Verdict	Evidence
1	<code>AuthContext.Provider</code> value is un-memoized inline object	✅ Correct	<code>src/auth/AuthProvider.tsx</code> line ~523: <code>const value: AuthContextValue = { user: userState, ... }</code> then <code><AuthContext.Provider value={value}></code> . No <code>useMemo</code> . Every render creates a new object → every <code>useAuth()</code> consumer re-renders.
2	<code>ProtectedRoute.tsx</code> unmounts the whole layout via <code>if (activePanel !== matched.panel) return <div>Loading...</div></code> and this is essentially a bug	⚠️ Partially correct, but the mechanism the summary describes doesn't exist	The real file is <code>src/app/guards/ProtectedRoute.tsx</code> . It does render a spinner instead of <code><Outlet/></code> while <code>activePanel !== matched.panel</code> , and the code comment even says this is <i>intentional</i> ("prevent rendering children with the wrong/stale user context during the transition"). But <code>activePanel</code> only changes when <code>switchPanel()</code> runs and actually passes a different panel value — see finding below, which is the real bug in this area and is not the one the summary names.
3	Flawed guard in <code>connectSocket(): if (socket?.connected) return;</code> creates a race because <code>socket.connected</code> is false during the async handshake, so a second call before the handshake completes drops the previous socket and creates a new one, causing a connect/disconnect loop	✅ Correct as a mechanism, but the summary's own quoted code and its narrative of why it fires repeatedly are both wrong	Real file <code>src/lib/socket.ts</code> line 15-16: <code>if (socket?.connected) return;</code> — identical guard, so the race is real. But the summary's quoted <code>socket.ts</code> (with <code>console.log('Socket connected')</code> , no <code>isConnecting</code> flag mention, framed as looping forever) doesn't match the file, and — more importantly — the real trigger isn't "repeated re-renders calling <code>connectSocket()</code> in a loop." It's multiple independent components calling <code>connectSocket()</code> in the same mount pass (see §3). This is a one-time-per-panel-load storm, not an infinite loop.
4	Polling intervals + socket-triggered refreshes cause excess requests; "similar patterns exist in <code>useKitchenDashboard</code> (30s) and others"	⚠️ Partially correct — real for Staff/Cleaning, false for Kitchen	<code>useStaffDashboard.ts</code> has a real <code>setInterval(scheduleRefresh, 5000)</code> (line 327) plus ~19 socket listeners that all call <code>scheduleRefresh()</code> . <code>usecleaning.ts</code> has the same pattern. But <code>features/kitchen/hooks/useKitchenDashboard.ts</code> contains no <code>setInterval</code> polling at all — it's purely socket-event-driven with a 200ms debounce (<code>setTimeout</code> , not <code>setInterval</code>). The Executive Summary's "30s interval in Kitchen" claim does not correspond to anything in the code.
5	React Query is bypassed; custom hooks fetch in <code>useEffect</code> and store in a client-side store instead	✅ Correct	<code>lib/queryClient.ts</code> configures a real <code>QueryClient</code> (<code>staleTime: 30s</code> , <code>refetchOnWindowFocus: false</code> — sane defaults), but <code>grep -rn "useQuery("</code> across <code>features/</code> returns zero matches. The client is wired up and never used. All dashboard hooks (<code>useStaffDashboard</code> , <code>useKitchenDashboard</code> ,

#	Claim	Verdict	Evidence
			<code>useCleaning</code> , etc.) hand-roll <code>apiClient.get</code> calls inside <code>useEffect</code> and push results into plain module-level stores (<code>staffStore</code> , <code>useKitchenStore</code> , etc.) instead.
6	Missing <code>useMemo</code> on context values is "minor... not the main bottleneck"	⚠️ Partially correct — actually more significant than the summary gives it credit for, because of a downstream bug the summary missed (see §3)	On its own, an unmemoized context causes extra renders, which is cheap. But because <code>switchPanel()</code> (called from inside <code>AuthProvider</code>) creates a brand-new <code>user</code> object on every route change, and at least 4 hooks/effects depend on <code>user</code> by reference rather than <code>user?.id</code> (see §3), the "minor" re-render cascade turns into repeated live API calls on ordinary navigation — which is one of the reported symptoms.
7	"The <code>SocketProvider</code> unconditionally calls <code>disconnect</code> on sign-out and reconnect on sign-in, which... can further exacerbate loops"	✅ Correct, and this compounds with §3	<code>SocketProvider.tsx</code> : <code>useEffect(() => { if (isAuthenticated) { connectSocket(); return () => disconnectSocket(); } disconnectSocket(); }, [isAuthenticated])</code> . This is expected behavior for auth transitions, but because <code>isAuthenticated</code> is derived from <code>userState && accessTokenState</code> and <code>userState</code> gets a new reference on unrelated navigations in some flows (not literally toggling <code>isAuthenticated</code> 's boolean value, so this specific effect is <i>not</i> re-triggered by the bug in §3 — <code>isAuthenticated</code> 's boolean value itself doesn't flip). This item is fine as stated but is not actually a major contributor once §3 is fixed.

3. The real primary root causes (not identified, or misidentified, in the Executive Summary)

● **Root cause A** — `switchPanel()` creates a new `user` object reference on *every route change*, even within the same panel

This is the most important finding and the Executive Summary never looked at it.

`ProtectedRoute.tsx`:

```
tsx

useEffect(() => {
  if (matched) {
    let shouldSwitchPanel = isPanelAuthenticated(matched.panel);
    ...
    if (shouldSwitchPanel) {
      switchPanel(matched.panel);
    }
  }
}, [location.pathname, matched, isPanelAuthenticated, switchPanel]);
```

This effect depends on `location.pathname`, so it re-runs on **every single navigation** — including navigating between two pages inside the *same* panel (e.g. `/staff/orders` → `/staff/tables`). Since the user is already authenticated for that panel, `shouldSwitchPanel` is `true` every time, so `switchPanel(matched.panel)` is called on every navigation, not just on an actual panel change.

`switchPanel()` in `AuthProvider.tsx`:

```
tsx

const switchPanel = useCallback((panel: Panel) => {
  setActivePanel(panel); // writes localStorage every nav (cheap)
  setActivePanelState(panel); // same string value → React bails out,
  const token = getAccessToken(panel);
  const stored = getStoredUser(panel);
  setAccessTokenStateRaw(token); // same string value → bails out
  setUserState(stored ? toAuthUser(stored, panel) : null); // 🚩 NEW OBJECT
}, []);
```

`toAuthUser()` builds a brand-new object literal every time it's called, even when every field is identical to the previous user object. React's `Object.is` bailout doesn't help here because the *reference* changed. So:

1. `AuthProvider` re-renders on every route change.
2. Its context `value` (Root cause noted in the table, item 1/6) is a fresh object every render, so **every one of the 30+ `useAuth()` consumers** across every layout, sidebar, and topbar re-renders on every navigation — not just at login.
3. Any hook that puts `user` (not `user?.id`) in a dependency array re-fires its data-fetching effect on every navigation. Confirmed instances:
 - `features/staff/hooks/useProfile.ts`: `const refresh = useCallback(async () => {...}, [user])` combined with `useEffect(() => { void refresh(); }, [refresh])` → `profileAPI.getProfile()` fires again on every route change while `StaffProfilePage` (or anything using this hook) is mounted.
 - `features/cleaning/hooks/useProfile.ts` — identical pattern.
 - `features/superAdmin/pages/EditProfile.tsx` line 386 — same `[user]` dependency.
 - `features/admin/components/settings/AdminSubscriptionCheckoutModal.tsx` line 141 — `[isOpen, restaurant, admin, user]`.

This directly explains the reported symptom "**Network requests increase simply by navigating**" — and it explains it precisely, for a mechanism the Executive Summary didn't investigate at all. It also explains "console logs appear multiple times" for any component with a `console.log` in its render body, since the re-render cascade touches nearly the whole authenticated tree on every click.

(Note: `SuperAdminLayout.tsx` has a superficially similar `useEffect(..., [user, syncThemeFromProfile])`, but it's correctly guarded with `user.id !== lastSyncedUserIdRef.current`, so it does **not** re-fire on the reference-only change — a good pattern that the other three hooks above should copy.)

Fix: `switchPanel()` should only update state (and only build a new `toAuthUser()` object) when the panel actually changes, or when the underlying stored user data has actually changed. Simplest correct fix: early-return if `panel === activePanel` and the stored token/user are unchanged from current state.

● **Root cause B** — every panel loads 3-4 independent components that each call `connectSocket()` on mount, in the same commit, before the first handshake resolves

This is the real mechanism behind the "flawed guard" — the guard itself is fine in isolation; the problem is *how many places call it simultaneously*.

For the Staff panel specifically, on a single page load of e.g. `/staff/dashboard`, these all mount together and each independently calls `connectSocket()` in their own `useEffect`:

- `SocketProvider` (once, at app root, on `isAuthenticated` becoming true)
- `StaffLayout.tsx` (line ~60: `connectSocket()`; inside its own big effect)
- `StaffSidebar` → `useStaffDashboard()` → `connectSocket()` (line 331 of `useStaffDashboard.ts`)
- `StaffTopBar` → `useStaffDashboard()` → `connectSocket()` (same hook, second instance)
- the routed page itself (e.g. `StaffDashboard.tsx`) → `useStaffDashboard()` → `connectSocket()` (third instance)

`io(...)` returns synchronously; the network handshake is async. Because React fires effects for siblings in the same commit synchronously, one after another, several of these `connectSocket()` calls execute **before** `socket.connected` becomes `true` for the first one. Each of those calls therefore fails the `if (socket?.connected) return;` guard and creates a brand-new `io(...)` instance, silently orphaning the previous one (never explicitly `.disconnect()`-ed — it just keeps trying to connect in the background with no listeners attached). Only the last-created socket ends up referenced by the module-level `socket` variable and receiving your `.on(...)` listener registrations.

This is what actually produces the "multiple connect/disconnect" churn visible in the Network/WS tab — a burst of 3-4 socket creations at the start of every panel load, not a sustained infinite loop. In dev, `React.StrictMode`'s double-effect-invocation adds one more full mount→unmount→mount cycle on top of this, which is why it can look even worse locally than in a production build.

Same pattern confirmed in other panels:

- `features/admin/pages/AdminDashboard.tsx`, `CustomersPage.tsx` — both call `connectSocket()` directly.
- `features/superAdmin/store/AlertsStore.ts`, `SuperadminDashboard.tsx` — both call `connectSocket()`.
- `features/cleaning/hooks/usecleaning.ts` — calls `connectSocket()`, and is used by `CleaningSidebar`, `CleaningTopBar`, and every Cleaning page (same multi-instance pattern as Staff).

`KitchenLayout.tsx`/`useKitchenDashboard.ts` are the exception — they rely solely on `SocketProvider`'s single connection and never call `connectSocket()` themselves, which is the correct pattern and is why Kitchen is comparatively less affected (contrary to the Executive Summary's claim that Kitchen has 30s polling — it doesn't).

Fix: `connectSocket()` should only ever be called from `SocketProvider`. Every other call site (`StaffLayout`, `useStaffDashboard`, `usecleaning`, `AdminDashboard`, `CustomersPage`, `AlertsStore`, `SuperadminDashboard`) should be deleted and replaced with `getSocket()` (read-only access), since the socket is guaranteed to already exist by the time any child of `SocketProvider` mounts.

● **Root cause C — the same hook (`useStaffDashboard`, `useCleaning`) is independently instantiated by 8-13 different components per panel**

`useStaffDashboard()` is called from `StaffTopBar`, `StaffSidebar`, and **every single staff page** (`StaffMonitorPage`, `StaffFoodReadyPage`, `StaffAlertsPage`, `StaffReportsPage`, `StaffOrdersPage`, `StaffTablesPage`, `StaffDashboard`, `StaffTableTurnoverPage`, `StaffMenuPage`, `StaffReservationsPage`, `StaffRequestsPage`) — 13 files total. `useCleaning()` follows the identical pattern (sidebar, topbar, every cleaning page).

Each mounted instance independently:

- registers its own `setInterval(scheduleRefresh, 5000)`
- registers its own copy of ~19 `socket.on(handleSync)` listeners bound to the same shared socket

The good news: `scheduleRefresh()`/`refreshDashboard()` use **module-level** dedup flags (`refreshScheduled`, `refreshInProgress`, `queuedRefresh`), so this doesn't multiply actual HTTP calls as badly as it looks — concurrent triggers within the same 200ms window collapse into one real fetch. But it's still real waste: on a typical Staff dashboard view (Sidebar + TopBar + page all using the hook simultaneously), you get 3 independent 5-second timers and ~57 registered socket listeners for the same 19 event names, all doing the same job. `StaffLayout.tsx` also independently sets up an overlapping (not identical) third set of listeners for many of the same events (`table.status.changed`, `order.created`, `order.updated`, `bill.requested`, `bill.paid`, plus its own 15s polling fallback), which is a fourth redundant subscription layer.

This architecture is legitimately wasteful and worth fixing, but it is **not** the primary driver of "requests increase on every navigation" (that's Root cause A) or of the socket churn (Root cause B). It's a real, secondary contributor to overall request volume and listener-registration overhead, and to CPU/battery cost on idle screens.

Fix: Hoist dashboard data-fetching + socket subscription to a single place per panel (the Layout, or a dedicated context/provider), and have pages/Sidebar/TopBar consume from a shared store/context instead of each calling the fetching hook independently.

4. Corrected root cause ranking

Impact	Root cause	Why
Critical	A. <code>switchPanel()</code> creates a new <code>user</code> object on every navigation, cascading through the unmemoized <code>AuthContext</code> and into any hook keyed on <code>user</code> by reference	Directly reproduces "requests increase just from navigating" and "console logs multiple times," on every click, in every panel — the widest-reaching bug found.
Critical	B. Multiple independent <code>connectSocket()</code> call sites fire in the same mount commit, racing the <code>if (socket?.connected)</code> guard and creating orphaned sockets	Directly reproduces the socket connect/disconnect churn on every panel load. Present in Staff, Admin, SuperAdmin, Cleaning, Customer — not Kitchen.
Medium	C. The same dashboard hook is instantiated 8-13× per panel, each with its own interval + full listener set	Real waste and listener bloat, but largely self-deduped by module-level flags; doesn't independently explain the reported symptoms.
Medium	Unmemoized <code>AuthContext</code> value	True on its own, and it's the mechanism that turns Root cause A into a whole-tree re-render cascade — fixing it caps the blast radius even before A is fixed.
Low	React Query configured but unused	Real, and worth doing eventually for caching/dedup, but not causal for the reported symptoms — the custom stores already dedup fetches at the module level.
Not confirmed	"ProtectedRoute unmounts the whole layout on every panel mismatch" as originally described	The loading-spinner branch is real but only fires during an actual panel transition, which — once Root cause A is fixed — happens rarely (login, first load, switching panels), not on ordinary navigation.
Not confirmed	" <code>useKitchenDashboard</code> has a 30s polling interval"	No <code>setInterval</code> exists anywhere in that file.

5. Implementation plan (safest order)

Each step is independently revertable. Suggested before/after metrics: socket connects per panel load, distinct API calls in the first 10s after login, API calls per minute while idle, API calls triggered per single in-panel navigation (should be 0 for pages not on that route), and console log count per navigation.

Step 1 — Fix `switchPanel()` to be a no-op when nothing actually changed (Critical, do first)

In `src/auth/AuthProvider.tsx`, short-circuit when the panel is already active and the stored token/user haven't changed, and when they have changed, only build a new `AuthUser` object if its fields actually differ (or accept the minor cost but stop calling this every navigation — see Step 2 for the real fix to *when* it's called). The more targeted fix is

in `ProtectedRoute.tsx`: only call `switchPanel(matched.panel)` when `matched.panel !== activePanel`, not unconditionally whenever `isPanelAuthenticated` is true:

```
diff
- if (shouldSwitchPanel) {
+ if (shouldSwitchPanel && matched.panel !== activePanel) {
    switchPanel(matched.panel);
  }
```

Verify: navigating between pages within one panel should produce zero `AuthProvider` re-renders and zero calls into `switchPanel`. Confirm with a temporary `console.count('switchPanel')`.

Step 2 — Memoize the `AuthContext` value (*Critical, do alongside Step 1*)

```
diff
- const value: AuthContextValue = { user: userState, ... };
+ const value = useMemo<AuthContextValue>(() => ({ user: userState, ... }), [
+   userState, accessTokenState, initializing, activePanel, /* stable callback
+ ]);
```

This caps the blast radius of any future reference-identity slip, independent of Step 1.

Verify: React DevTools Profiler shows `useAuth()` consumers re-rendering only when `user`/`accessToken`/`activePanel`/`initializing` actually change.

Step 3 — Fix the `[user]`-keyed effects to depend on `user?.id` (or specific fields), not the object (*High*)

Apply to `features/staff/hooks/useProfile.ts`, `features/cleaning/hooks/useProfile.ts`, `features/superAdmin/pages/EditProfile.tsx`, `AdminSubscriptionCheckoutModal.tsx`. Mirror the pattern already used correctly in `SuperAdminLayout.tsx`

(`lastSyncedUserIdRef`). **Verify:** with Steps 1–2 in place this becomes defense-in-depth rather than a live bug, but it should be fixed regardless since any future re-render source would otherwise re-trigger profile fetches.

Step 4 — Centralize `connectSocket()` to `SocketProvider` only (*Critical*)

Remove the `connectSocket()` calls from `StaffLayout.tsx`, `useStaffDashboard.ts`, `usecleaning.ts`, `CustomerLayout.tsx`, `features/customer/store/customer.store.ts`'s redundant reconnect-on-set (keep the intentional forced-reconnect-on-new-session, that one's deliberate), `admin/pages/AdminDashboard.tsx`, `admin/pages/CustomersPage.tsx`, `superAdmin/store/AlertsStore.ts`, `superAdmin/pages/SuperadminDashboard.tsx`. Replace with `getSocket()` (they can assume it exists, since they only render once authenticated, i.e. after `SocketProvider` has already connected). **Verify:** exactly one `[Socket] Connected` log per panel load in the Network/WS tab (two in dev under StrictMode — that's expected, not a bug).

Step 5 — Harden `connectSocket()`'s guard defensively (*High, cheap insurance*)

Even after Step 4, keep a stronger guard so any future accidental double-call can't create an orphaned socket:

```
ts
```



```

let isConnecting = false;
export function connectSocket(): void {
  if (socket?.connected || isConnecting) return;
  isConnecting = true;
  ...
  socket.once('connect', () => { isConnecting = false; ... });
  socket.once('connect_error', () => { isConnecting = false; ... });
}

```

Verify: manually call `connectSocket()` twice synchronously in dev tools console — should only ever create one live socket.

Step 6 — Consolidate the redundant dashboard-hook instantiation (*Medium*)

Move the data-fetching + socket-subscription responsibility for Staff/Cleaning dashboards to a single place per panel (the Layout or a dedicated context), and have `(Sidebar)/(TopBar)/pages` read from a shared store instead of each independently calling `useStaffDashboard()`/`useCleaning()`. This doesn't fix a live symptom on its own (thanks to the existing module-level dedup) but removes ~50+ redundant listener registrations per panel and the timer overhead. **Verify:** listener count on the socket object (`(socket._callbacks)` in dev tools, or add a temporary counter) drops from ~57 to ~19 for a given event set.

Step 7 (optional, lower priority) — Adopt React Query for the dashboard hooks

Since the `(QueryClient)` is already correctly configured (`(staleTime: 30s)`, `(refetchOnWindowFocus: false)`) but unused, migrating `(useStaffDashboard)/(useKitchenDashboard)/(useCleaning)` to `(useQuery)` would give real caching/deduplication for free and let Step 6 be largely automatic (multiple components calling the same query key dedupe natively). This is a larger refactor and should come after Steps 1–5 have removed the acute symptoms.

6. Bottom line

The Executive Summary correctly identified that the problem is systemic client-side architecture rather than one panel or one API, and correctly flagged the unmemoized `(AuthContext)` and the bypassed React Query as real (if secondary) issues. But its two headline claims don't hold up against the actual code:

- The quoted `(ProtectedRoute.tsx)/(socket.ts)` snippets don't match what's in the repository (they may be from a different branch, an LLM reconstruction, or a misread).
- The "30s Kitchen polling" claim doesn't correspond to any code in the repo.

The actual primary drivers are two bugs the summary never investigated:

1. `(switchPanel())` is invoked on every navigation (not just real panel switches) and always builds a fresh `(user)` object, which — combined with the unmemoized context — cascades into a whole-app re-render and re-triggers any `([user])`-keyed data fetch on every click.
2. Multiple components independently call `(connectSocket())` in the same initial-mount commit, racing the (otherwise reasonable) `(if (socket?.connected))` guard and creating orphaned sockets on every panel load.

Fixing #1 and #2 (Steps 1–5 above) should eliminate the reported symptoms; Steps 6–7 are cleanup for efficiency, not correctness.